A Novel Memristor-Based Majority Logic and Efficient Approximate Full Adder for Image Processing

Zhouchao Gan[1†], Yifeng Xiong[1†], Fan Yang[1], Mengjie Li[1], Xiangshui Miao[1,2], Xingsheng Wang[1,2*]
[1]*School of Integrated Circuits & Wuhan National Lab for Optoelectronics, Huazhong University of Science and Technology, Wuhan, China*
[2]*Hubei Yangtze Memory Labs, Wuhan, China*
[*]*E-mail: xswang@hust.edu.cn*

2025 IEEE International Symposium on Circuits and Systems (ISCAS) | 979-8-3503-5683-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/ISCAS56072.2025.11044245

***Abstract*—The memristor, an emerging non-volatile memory device, is well-suited for in-memory computing (IMC) due to its capability to simultaneously store data and perform logic operations. Majority (MAJ) logic is a type of expressive Boolean logic that performs well in logic synthesis. This paper proposes a novel MAJ logic implementation scheme to improve the latency of n-bit full adder (FA) from 6n+1 to 5n+1. Additionally, an approximate computing approach is introduced, yielding an n-bit approximate adder with a significantly reduced delay of 2n+1, at the cost of precision. Both exact and approximate adders were verified experimentally, and the error quality metrics of the approximate design were thoroughly evaluated. Furthermore, hybrid configurations combining exact and approximate adders were applied to image processing tasks, where the peak signal-to-noise ratio (PSNR) for most hybrid designs remained within an acceptable range (>30dB).**

Keywords*—*In-Memory Computing, Memristor, Majority logic, Approximate Computing, Image Processing.

I. Introduction

In-memory computing (IMC) offers higher bandwidth and improved energy efficiency for data-intensive applications by enabling logical operations to be performed directly in memory, which has the potential to break through the von Neumann bottleneck [1]. Several emerging devices have demonstrated the capability to implement IMC, with memristors gaining notable attention due to their high speed, density, and CMOS compatibility [2]. Various logic primitives have been implemented based on memristors [3]–[10], and significant efforts have been made to improve computational efficiency [11]–[16]. IMC logic can be categorized into two major types based on the implementation mechanism: WRITE-based and READ-based. The WRITE-based logic schemes, such as IMPLY and MAGIC, rely on switching the states of memristors during computation [3], [4]. Conversely, READ-based logic evaluates the logic without altering the memory state; the logic result is derived from the output of the sense amplifier [7], [16]–[18].

Addition is the most fundamental operation within the arithmetic units of digital circuits [19]. Enhancing the efficiency of addition relies heavily on the use of optimized logic primitives. The Majority-Inverter Graph (MIG) is a complete and expressive logic primitive that provides superior logical optimization compared to traditional AND/OR/INVERTER Graphs (AOIGs) [20], [21]. There are two approaches to implementing the three-input majority (MAJ) gate in MIG logic using memristors [5], [22], which are WRITE-based and READ-based respectively. Both approaches offer distinct advantages and trade-offs. Further exploration of MAJ gate mapping is essential to fully exploit the inherent efficiency of MIG logic.

Approximate computing (AC), as an emerging paradigm, effectively reduces computational complexity and optimizes hardware and latency performance by tolerating minor accuracy loss [23]. AC is particularly well-suited for error-tolerant applications, including image processing, pattern recognition, and machine learning [23]–[25]. In image processing, for instance, the quality degradation from controlled precision loss is often imperceptible to human vision [26]. The integration of approximate algorithms into WRITE-based IMPLY logic has led to substantial improvements in computational efficiency [27]–[30]. Nevertheless, the potential of AC within READ-based IMC logic remains largely unexplored.

This paper proposes a novel MAJ logic to improve the efficiency of addition operations at the primitive level. The implementation of approximate addition is further investigated, with its performance evaluated in the context of image processing. The rest of this paper is organized as follows. Section II presents the proposed MAJ logic and experiment results. Section III introduces the design and testing results of 1-bit exact and approximate full adders, along with the analysis of overhead for n-bit full adders. The application of hybrid-precision addition in image processing is examined in Section IV, where image quality metrics under varying precision levels are evaluated. Finally, Section V concludes the paper.

II. Proposed Majority Logic

A. Device Characteristics

Memristors can switch between high and low resistance states in response to applied voltage. Fig. 1 (a) presents a cross-sectional TEM image of a 1-transistor-1-memristor (1T1R) integrated unit of a $T_i/HfO_2/T_iN$ memristor device used to demonstrate the logic scheme and addition

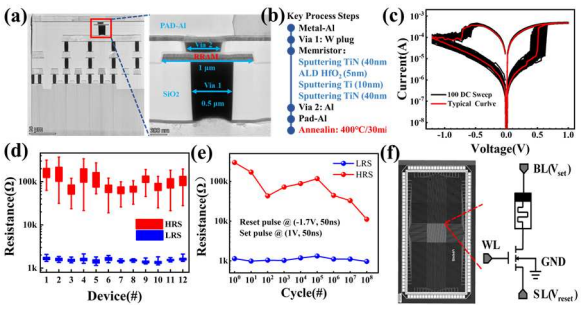
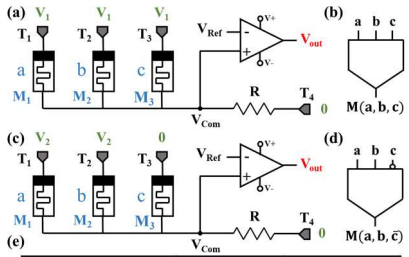


Fig. 1. (a) TEM image of the cross-sectional cut and (b) key manufacturing steps of the 1T1R integrated unit. (c) Measured resistive I-V switching behavior in DC sweeping mode. (d) Measured HRS and LRS distributions of 12 devices. (e) More than 10^8 cycling endurance under voltage pulses. (f) The metallographic electron microscopy image of the 32×32 1T1R array.

[†]These authors contributed equally. [*]Corresponding author

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:08:12 UTC from IEEE Xplore. Restrictions apply.



Physical variable	Logic "0"	Logic "1"
Input: R_M	HRS	LRS
Output : V_{out}	V-	V+

Fig. 2. Circuit implementation of (a) $M(a, b, c)$ logic and (c) $M(a, b, \bar{c})$ logic. Simplified diagram of $M(a, b, c)$ logic and (c) $M(a, b, \bar{c})$ gates. (e) Definition of logical inputs and outputs.

functionality. The key fabrication steps of the device are summarized in Fig. 1 (b). In Fig. 1 (c), the resistive switching characteristics of the device over 100 consecutive DC cycles are illustrated. Fig. 1 (d) provides a statistical box plot of the high resistance state (HRS) and low resistance state (LRS) distributions of 12 devices under 100 consecutive pulses, showing a large switching window and a uniform low-resistance distribution. Fig. 1 (e) demonstrates that the memristor can endure 10^8 cycles under pulsed operation while maintaining a resistance state window of approximately tenfold. The above device characteristics are advantageous for the reliable execution of logic operations. Fig. 1 (f) displays a metallographic electron microscope image of a 32 × 32 1T1R array, alongside a schematic of the 1T1R structure, where the BL terminal is connected to the positive electrode of the memristor, a notation used consistently in the following sections.

B. Principle of the Proposed MAJ Logic

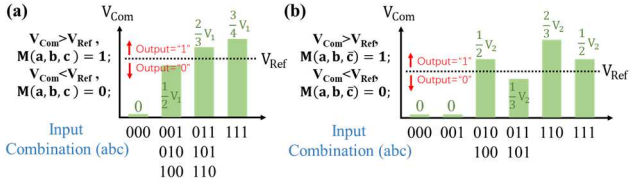


Fig. 3. The main ideas of (a) $M(a, b, c)$ and (b) $M(a, b, \bar{c})$ logic.

As a complete logic primitive, MIG logic consists of three-input MAJ gates and inverters (INVs), which have been demonstrated to be more efficient in logic optimization compared to traditional AOIGs [20], [21], [31]. The expressions for the MAJ and INV logic are as follows:

$$M(a, b, c) = a \cdot b + b \cdot c + a \cdot c \quad (1)$$

$$INV(a) = \bar{a} \quad (2)$$

There are currently two primary methods for implementing MAJ logic based on memristors. In [5], a WRITE-based $M(a, b, \bar{c})$ logic is achieved by encoding the voltage across the memristor and its resistance state within a single device. In contrast, [22] realizes $M(a, b, c)$ logic by selecting three memristors for read operations, improving the input destructiveness in [5] and increasing the parallelism of calculations in the array, although more memristors were required. However, the INV logic relies on external inverter circuits [22], which to some extent limits computational efficiency.

This paper proposes a reconfigurable memristor-based circuit for implementing both $M(a, b, c)$ and $M(a, b, \bar{c})$ logic operations. As shown in Fig. 2 (a), the $M(a, b, c)$ logic is

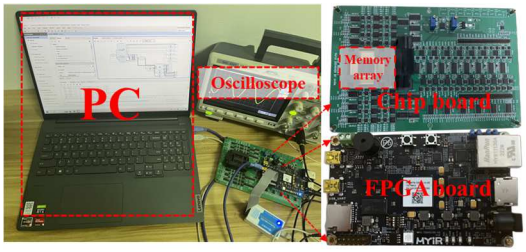


Fig. 4. Experiment test platform.

realized by applying voltages $(V_1, V_1, V_1, 0)$ to the control terminals (T_1, T_2, T_3, T_4) in a three-memristor-one-resistor (3M1R) structure. The logic inputs (a, b, c) correspond to the resistance value R_M of the memristors (M_1, M_2, M_3), while the logic result is retrieved by sampling the voltage of the common node (V_{Com}) in the 3M1R structure through a comparator, as summarized in Fig. 2 (e). The underlying principle is that when at least two of the three inputs memristors (M_1, M_2, M_3) are in a LRS, V_{Com} will be higher than when at most one memristor is in the LRS. The resistance of the series resistor R is equal to the low resistance R_{on} of the memristor. The main idea for designing $M(a, b, c)$ logic is illustrated in Fig. 3 (a). To execute the $M(a, b, c)$ logic correctly, the reference voltage V_{Ref} of the comparator must satisfy the following equation:

$$\frac{V_1}{2} < V_{Ref} < \frac{2V_1}{3} \quad (3)$$

To implement $M(a, b, \bar{c})$ logic, only the voltage of the control terminal (T_1, T_2, T_3, T_4) needs to be changed to $(V_2, V_2, 0, 0)$. Since the control terminal T_3 of the memristor M_3 is connected to a low level, it has the effect of pulling down the voltage of V_{Com}, and is therefore expressed as an inverse variable in the logical expression. The design principle for $M(a, b, \bar{c})$ is depicted in Fig. 3 (b), where V_{Ref} must satisfy the condition:

$$\frac{V_2}{3} < V_{Ref} < \frac{V_2}{2} \quad (4)$$

Therefore, by merely configuring the operating voltages during readout, both $M(a, b, c)$ and $M(a, b, \bar{c})$ logic operations can be executed using the same circuit structure. The corresponding schematics for the two logic gates are provided in Fig. 2 (b) and (d), respectively.

C. Experimental Setup

To verify the correctness of the proposed logic, we established a test platform, as illustrated in Fig. 4. To mitigate the effects of parasitic capacitance, a pre-charging method was employed during testing. In the 1T1R structure depicted in Fig. 1(f), a DC voltage was applied to both the bit line (BL) and the source line (SL), while a pulse with a width of 1 µs was applied to the word line (WL). The control voltage applied to the memristor array is generated by the FPGA board controlling the DAC in the chip board. During the

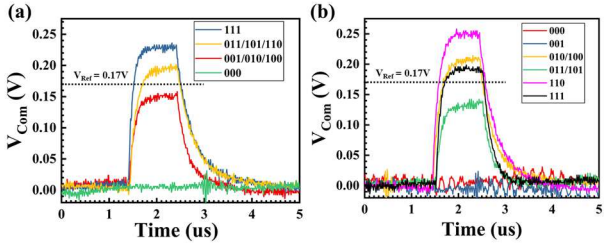


Fig. 5. The test waveforms of (a) $M(a, b, c)$ logic and (b) $M(a, b, \bar{c})$ logic.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:08:12 UTC from IEEE Xplore. Restrictions apply.

experiments, V_1 = 0.3V and V_2 = 0.4 V. The test results, as shown in Fig. 5, indicate that by setting the reference voltage V_{Ref} = 0.17 V, both $M(a, b, c)$ and $M(a, b, \bar{c})$ logic operations were successfully executed.

III. EXACT AND APPROXIMATE FULL ADDERS

A. *Proposed Exact 1-bit Full Adder*

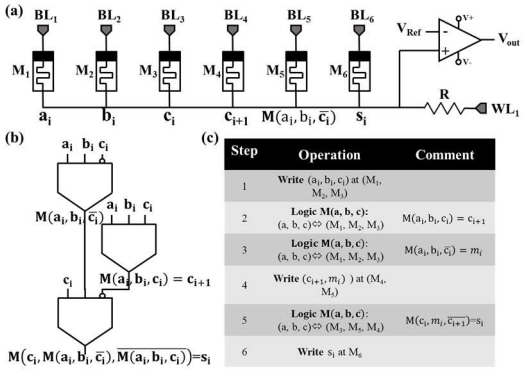


Fig. 6. 1-bit exact addition: (a) Circuit diagram, (b) Gate-level representation, and (c) Detailed implementation steps.

For a 1-bit exact full adder (FA) with three inputs (the addends a_i, b_i, and the carry-in c_i) and two outputs (the sum s_i and the carry-out c_{i+1}), its functionality can be expressed using MIG logic as follows:

$$c_{i+1} = M(a_i, b_i, c_i) \tag{5}$$

$$s_i = M(c_i, M(a_i, b_i, \bar{c_i}), \overline{M(a_i, c_i, b_i)}) \tag{6}$$

By iterating the proposed $M(a, b, c)$ and $M(a, b, \bar{c})$ logic, the functionality of a 1-bit exact FA can be implemented within the circuit structure shown in Fig. 6 (a). Fig. 6 (b) presents the gate-level schematic of the exact 1-bit FA, and the detailed implementation steps are outlined in Fig. 6 (c). Through sequential execution of $M(a, b, c)$, $M(a, b, \bar{c})$ and WRITE operations, the carry-out c_{i+1} is stored in memristor M_4 at step four, and the sum s_i is stored in memristor M_6 at step six.

B. *Proposed Approximate 1-bit Full Adder*

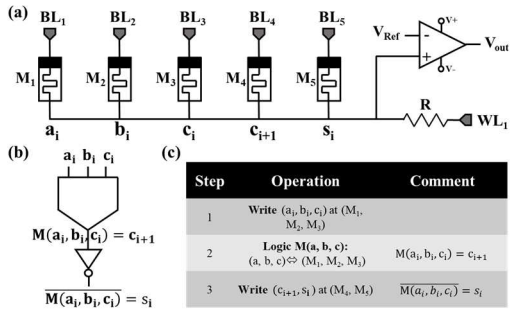


Fig. 7. 1-bit approximate addition: (a) Circuit diagram, (b) Gate-level representation, and (c) Detailed implementation steps.

By simplifying the logic expressions, approximate computing enables the full adder (FA) function to be executed with higher efficiency. The 1-bit approximate adder is defined by the following formulas:

$$c_{i+1} = M(a_i, b_i, c_i) \tag{7}$$

$$s_i = \overline{M(a_i, c_i, b_i)} \tag{8}$$

In this approximate design, the carry-out c_{i+1} remains identical to that of the exact adder, while the sum s_i is the inverse variable of c_{i+1}. As summarized in TABLE I, the carry-out c_{i+1} in the approximate adder is accurate, but the sum s_i deviates from the exact result for two input combinations. Fig. 7 (a) presents the circuit diagram for the 1-bit approximate adder, and the gate-level representation is shown in Fig. 7 (b). The 1-bit approximate adder requires only 5 memristors and 3 steps for execution, as outlined in Fig. 7 (c).

TABLE I. THE TRUTH TABLE OF THE EXACT FULL-ADDER AND THE PROPOSED APPROXIMATE FULL-ADDER

a_i	b_i	c_i	Exact s_i	Exact c_{i+1}	Approximate s_i	Approximate c_{i+1}
0	0	0	0	0	1×	0√
0	0	1	1	0	1√	0√
0	1	0	1	0	1√	0√
0	1	1	0	1	0√	1√
1	0	0	1	0	1√	0√
1	0	1	0	1	0√	1√
1	1	0	0	1	0√	1√
1	1	1	1	1	0×	1√

C. *n-bit Exact and Approximate Full Adder*

For multi-bit addition, the ripple carry adder (RCA) architecture is commonly preferred due to its structural simplicity and ease of timing control. Extending the proposed 1-bit precise adder to an n-bit adder allows the operands to be written in a single step. Consequently, the delay for an n-bit exact addition is $5n$+1 steps. Without considering repeated erasing and writing of multiplexed memristors, 4n memristors are needed in an n-bit exact adder to store a_0-a_{n-1}, b_0-b_{n-1}, s_0 - s_{n-1} and the intermediate results m_0 - m_{n-1}, n+1 memristors to store c_0-c_n, leading to a total device overhead of 5n+1. It can also be deduced that n-bit approximate addition requires 2n+1 steps and 4n+1 memristors.

TABLE II summarizes the overhead of various n-bit exact FA and approximate FA. The results show that, thanks to the proposed reconfigurable logic, both the delay and memoristor overhead of the exact adder based on MIG logic are reduced from 6n+1 to 5n+1. When using the approximation algorithm, the computational efficiency of n-bit addition is more significantly improved (the delay is reduced by 2/3 and the area is reduced by 1/3). Compared to the IMPLY-based approximation scheme, the proposed n-bit approximate addition offers superior speed, albeit at the cost of higher device consumption.

TABLE II. COMPARISON BETWEEN THE PROPOSED ADDERS AND THE STATE-OF-THE-ART (SOA)

Ref	Logic type	Algorithm	No. of Memristors	No. of Steps
[11]	IMPLY	Exact Serial	2n+3	22n
[12]		IMPLY Semi-Serial	2n+6	10n+2
[30]		SINC	**2n+1**	3n
		SINC+	2n+2	3n+3
[16]	MIG	Exact Serial	6n+1	6n+1
This work		Exact Serial	5n+1	5n+1
This work		Approximate Serial	4n+1	**2n+1**

D. *Experimental Verification of Full Adder*

In the test platform illustrated in Fig. 4, we experimentally validated the proposed exact and approximate 1-bit full

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:08:12 UTC from IEEE Xplore. Restrictions apply.

adders. The results, as shown in Fig. 8, confirm the correct operation of both adders.

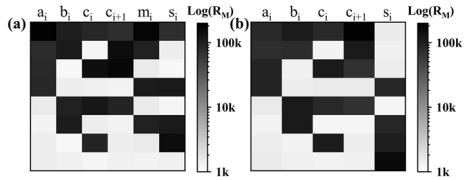


Fig. 8. (a) Exact and (b) approximate 1-bit full adder test results.

IV. APPLICATION IN IMAGE PROCESSING

A. Error Analysis

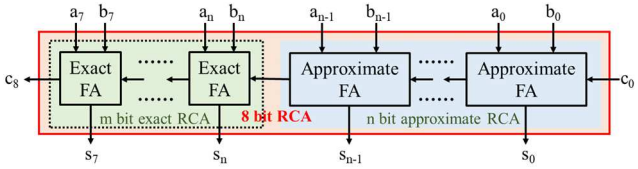


Fig. 9. A schematic diagram of the 8-bit RCA structure with varying degrees of mixed exact and approximate addition.

Approximate computing is well-suited for applications that are error-tolerant, most commonly such as image processing [23]. In this paper, we use image addition, a fundamental operation for tasks such as image masking [32], as a case study. Image addition involves adding the pixel values at corresponding positions in two images. For example, the addition of two 512×512 8-bit grayscale images can be performed using an 8-bit adder. As shown in Fig. 9, the addition is executed through a mixed-precision approach, where an approximate ripple carry adder (RCA) is used for the lower n bits and an exact RCA for the upper m bits (where $m + n = 8$). For simplicity, this configuration is denoted as "m Exa. +n Approx.". Error metrics such as Mean Error Distance (MED), Normalized Mean Error Distance (NMED), and Mean Relative Error Distance (MRED), are employed to evaluate and compare the error behavior of approximate adders [33], [34]. Detailed descriptions of these metrics are provided in [30].

We comprehensively evaluate all possible combinations of addends for each addition configuration using Python, and the results are presented in TABLE III. Given the accuracy of the carry in the approximate addition, the error metrics demonstrate relatively low values, which are further reflected in the subsequent image processing tasks.

B. Application-level Simulation

To assess the quality of approximate calculations in image processing, commonly used metrics include Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index (SSIM), and Mean Structural Similarity Index (MSSIM). PSNR reflects the noise level in the image, with values above 30 dB generally deemed acceptable [35]. SSIM quantifies the similarities and differences between two images of the same dimensions. For a more detailed explanation of SSIM and MSSIM, please refer to [36], [37].

Fig. 10 illustrates an example of image addition using both exact and mixed-precision addition techniques. The corresponding image quality metrics are computed via a Python program and summarized in TABLE IV. The results indicate that even when the lower five bits of the 8-bit RCA are processed using an approximate adder, the simulated image addition still meets acceptable quality standards (PSNR > 30 dB).

TABLE III. ERROR METRICS OF 8-BIT RCA WITH DIFFERENT APPROXIMATION DEGREES

m Exa. +n Approx.	**8-bit RCA**		
(m, n)	MED	NMED	MRED
(7, 1)	0.25	0.00049	0.00139
(6, 2)	0.75	0.00147	0.00419
(5, 3)	1.72	0.00336	0.00974
(4, 4)	3.62	0.00708	0.02095
(3, 5)	7.38	0.01444	0.04432
(2, 6)	14.86	0.02909	0.09508
(1, 7)	29.81	0.05835	0.21157
(0, 8)	59.69	0.11682	0.4956

TABLE IV. IMAGE ADDITION SIMULATION AND ITS IMAGE QUALITY METRICS

(m, n): m Exa. +n Approx.	**PSNR (dB)**	**SSIM**	**MSSIM**
(7, 1)	54.2889	0.998	0.9979
(6, 2)	48.0908	0.9951	0.9951
(5, 3)	43.098	0.9848	0.9847
(4, 4)	36.6299	0.9596	0.9594
(3, 5)	30.2222	0.9166	0.9156
(2, 6)	23.995	0.8693	0.8682

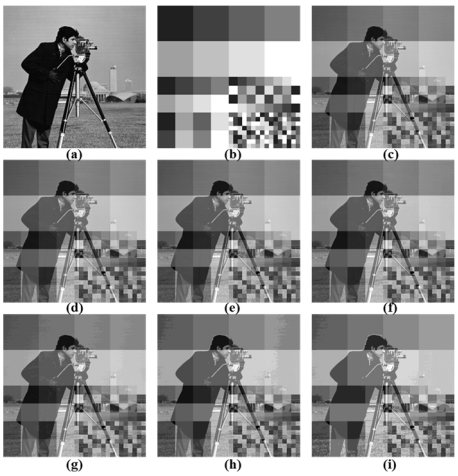


Fig. 10. Image addition example: Image addition example: (a) camera, (b) mask, (c) exact image addition, (d-i) increasing approximation from "7 Exa. +1 Approx." to "2 Exa. +6 Approx.".

V. CONCLUSION

This paper presents a memristor-based reconfigurable majority gate logic, which significantly enhances the efficiency of arithmetic operations. By reducing the latency of n-bit adders from 6n+1 to 5n+1, the proposed design enables more efficient computation. Furthermore, an approximate adder with a latency of 2n+1 is introduced, offering a substantial improvement in computational speed with minimal precision loss. The approximate computing approach demonstrates excellent performance in image processing, easily achieving a peak signal-to-noise ratio (PSNR) above 30 dB, highlighting its potential for fault-tolerant computing.

ACKNOWLEDGMENT

This work is supported in part by National Natural Science Foundation of China under Grants U2341221 and 62274070, Hubei Province Science and Technology Major Project under Grant 2022AEA001, Interdisciplinary Research Program of HUST (2023JCYJ042), and Hubei Key Laboratory of Advanced Memories.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:08:12 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Z. Sun, S. Kvatinsky, X. Si, A. Mehonic, Y. Cai, and R. Huang, "A full spectrum of computing-in-memory technologies," *Nat Electron*, pp. 1–13, Nov. 2023,

[2] H.-S. P. Wong and S. Salahuddin, "Memory leads the way to better computing," *Nature Nanotech*, vol. 10, no. 3, pp. 191–194, Mar. 2015,

[3] J. Borghetti, G. S. Snider, P. J. Kuekes, J. J. Yang, D. R. Stewart, and R. S. Williams, "'Memristive' switches enable 'stateful' logic operations via material implication," *Nature*, vol. 464, no. 7290, pp. 873–876, Apr. 2010,

[4] S. Kvatinsky *et al.*, "MAGIC—Memristor-Aided Logic," *IEEE Trans. Circuits Syst. II*, vol. 61, no. 11, pp. 895–899, Nov. 2014,

[5] P.-E. Gaillardon *et al.*, "The Programmable Logic-in-Memory (PLiM) computer," in *2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, Mar. 2016, pp. 427–432.

[6] Z. Sun, E. Ambrosi, A. Bricalli, and D. Ielmini, "Logic Computing with Stateful Neural Networks of Resistive Switches," *Adv. Mater.*, vol. 30, no. 38, p. 1802554, Sep. 2018,

[7] M. R. H. Rashed, S. Thijssen, S. K. Jha, F. Yao, and R. Ewetz, "STREAM: Towards READ-based In-Memory Computing for Streaming based Data Processing," in *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*, Jan. 2022, pp. 690–695.

[8] L. Xie *et al.*, "Scouting Logic: A Novel Memristor-Based Logic Design for Resistive Computing," in *2017 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, Jul. 2017, pp. 176–181.

[9] K. Alhaj Ali *et al.*, "Memristive Computational Memory Using Memristor Overwrite Logic (MOL)," *IEEE Trans. VLSI Syst.*, vol. 28, no. 11, pp. 2370–2382, Nov. 2020,

[10] J. Reuben and S. Pechmann, "A Parallel-friendly Majority Gate to Accelerate In-memory Computation," in *2020 IEEE 31st International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, Jul. 2020, pp. 93–100.

[11] S. G. Rohani and N. TaheriNejad, "An improved algorithm for IMPLY logic based memristive Full-adder," in *2017 IEEE 30th Canadian Conference on Electrical and Computer Engineering (CCECE)*, Apr. 2017, pp. 1–4.

[12] N. TaheriNejad, T. Delaroche, D. Radakovits, and S. Mirabbasi, "A Semi-Serial Topology for Compact and Fast IMPLY-based Memristive Full Adders," in *2019 17th IEEE International New Circuits and Systems Conference (NEWCAS)*, Jun. 2019, pp. 1–4.

[13] D. Radakovits, N. TaheriNejad, M. Cai, T. Delaroche, and S. Mirabbasi, "A Memristive Multiplier Using Semi-Serial IMPLY-Based Adder," *IEEE Trans. Circuits Syst. I*, vol. 67, no. 5, pp. 1495–1506, May 2020,

[14] N. Kaushik and B. Srinivasu, "Energy Efficient Memristor-Based Subtractors and Comparator for In-Memory Computing in MAGIC," *IEEE Trans. Circuits Syst. II*, pp. 1–1, 2023,

[15] Y. Wang *et al.*, "A Few-Step and Low-Cost Memristor Logic Based on MIG Logic for Frequent-Off Instant-On Circuits in IoT Applications," *IEEE Trans. Circuits Syst. II*, vol. 66, no. 4, pp. 662–666, Apr. 2019,

[16] V. Lakshmi, J. Reuben, and V. Pudi, "A Novel In-Memory Wallace Tree Multiplier Architecture Using Majority Logic," *IEEE Trans. Circuits Syst. I*, vol. 69, no. 3, pp. 1148–1158, Mar. 2022,

[17] K. Alhaj Ali *et al.*, "Memristive Computational Memory Using Memristor Overwrite Logic (MOL)," *IEEE Trans. VLSI Syst.*, vol. 28, no. 11, pp. 2370–2382, Nov. 2020,

[18] L. Xie *et al.*, "Scouting Logic: A Novel Memristor-Based Logic Design for Resistive Computing," in *2017 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, Jul. 2017, pp. 176–181.

[19] V. Gupta, D. Mohapatra, S. P. Park, A. Raghunathan, and K. Roy, "IMPACT: IMPrecise adders for low-power approximate computing," in *IEEE/ACM International Symposium on Low Power Electronics and Design*, Aug. 2011, pp. 409–414.

[20] L. Amaru, P.-E. Gaillardon, and G. De Micheli, "Majority-Inverter Graph: A New Paradigm for Logic Optimization," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 35, no. 5, pp. 806–819, May 2016,

[21] M. Soeken, L. G. Amaru, P.-E. Gaillardon, and G. De Micheli, "Exact Synthesis of Majority-Inverter Graphs and Its Applications," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 36, no. 11, pp. 1842–1855, Nov. 2017,

[22] J. Reuben, "Binary Addition in Resistance Switching Memory Array by Sensing Majority," *Micromachines*, vol. 11, no. 5, p. 496, May 2020,

[23] W. Liu, F. Lombardi, and M. Shulte, "A Retrospective and Prospective View of Approximate Computing [Point of View}," *Proc. IEEE*, vol. 108, no. 3, pp. 394–399, Mar. 2020,

[24] H. Jiang, C. Liu, L. Liu, F. Lombardi, and J. Han, "A Review, Classification, and Comparative Evaluation of Approximate Arithmetic Circuits," *J. Emerg. Technol. Comput. Syst.*, vol. 13, no. 4, pp. 1–34, Oct. 2017,

[25] S. E. Fatemieh, S. S. Farahani, and M. R. Reshadinezhad, "LAHAF: Low-power, area-efficient, and high-performance approximate full adder based on static CMOS," *Sustainable Computing: Informatics and Systems*, vol. 30, p. 100529, Jun. 2021,

[26] V. Gupta, D. Mohapatra, A. Raghunathan, and K. Roy, "Low-Power Digital Signal Processing Using Approximate Adders," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 32, no. 1, pp. 124–137, Jan. 2013,

[27] S. E. Fatemieh, M. R. Reshadinezhad, and N. TaheriNejad, "Approximate In-Memory Computing using Memristive IMPLY Logic and its Application to Image Processing," *2022 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 3115–3119, May 2022,

[28] S. E. Fatemieh, M. R. Reshadinezhad, and N. TaheriNejad, "Fast and Compact Serial IMPLY-Based Approximate Full Adders Applied in Image Processing," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 13, no. 1, pp. 175–188, Mar. 2023,

[29] F. Seiler and N. TaheriNejad, "An IMPLY-based Semi-Serial Approximate In-Memristor Adder," in *2023 IEEE Nordic Circuits and Systems Conference (NorCAS)*, Oct. 2023, pp. 1–7.

[30] F. Seiler and N. TaheriNejad, "Accelerated Image Processing Through IMPLY-Based NoCarry Approximated Adders," *IEEE Trans. Circuits Syst. I*, vol. 71, no. 11, pp. 5141–5154, Nov. 2024,

[31] V. Pudi, K. Sridharan, and F. Lombardi, "Majority Logic Formulations for Parallel Adder Designs at Reduced Delay and Circuit Complexity," *IEEE Trans. Comput.*, vol. 66, no. 10, pp. 1824–1830, Oct. 2017,

[32] H. A. F. Almurib, T. N. Kumar, and F. Lombardi, "Inexact designs for approximate low power addition by cell replacement," in *2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, Mar. 2016, pp. 660–665. Accessed: Oct. 12, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/7459392

[33] F. Seiler and N. TaheriNejad, "An IMPLY-based Semi-Serial Approximate In-Memristor Adder," in *2023 IEEE Nordic Circuits and Systems Conference (NorCAS)*, Oct. 2023, vol. 108, pp. 1–7.

[34] S. E. Fatemieh, M. R. Reshadinezhad, and N. TaheriNejad, "Fast and Compact Serial IMPLY-Based Approximate Full Adders Applied in Image Processing," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 13, no. 1, pp. 175–188, Mar. 2023,

[35] S. Mittal, "A Survey of Techniques for Approximate Computing," *ACM Comput. Surv.*, vol. 48, no. 4, pp. 1–33, May 2016,

[36] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE Trans. on Image Process.*, vol. 13, no. 4, pp. 600–612, Apr. 2004,

[37] H. Waris, C. Wang, W. Liu, and F. Lombardi, "AxSA: On the Design of High-Performance and Power-Efficient Approximate Systolic Arrays for Matrix Multiplication," *J Sign Process Syst*, vol. 93, no. 6, pp. 605–615, Jun. 2021,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:08:12 UTC from IEEE Xplore. Restrictions apply.